

C语言指针与链表专项训练 · Day 1

参考：谭浩强《C程序设计》

题型：单项选择题

题量：50题

重点：指针、字符串指针、动态内存、单链表

答题说明

- 每题只有一个正确答案。
- 建议在不查看资料和答案的情况下完成。
- 推荐用时：50~70分钟。
- 答题格式示例： 1B 2C 3A 4D
- 不确定的题可以标记为： 12B? 或 12?

一、指针基础

1. 定义一个指向整型变量的指针，正确的是：

- A. `int p;`
- B. `int *p;`
- C. `*int p;`
- D. `pointer int p;`

2. 执行下面代码后，a 的值是：

```
int a = 10;  
int *p = &a;  
*p = 20;
```

- A. 10
- B. 0
- C. 20

- D. 不确定

3. 运算符 & 用在变量前面的作用是：

- A. 获取变量的地址
- B. 获取指针指向的值
- C. 释放变量
- D. 将变量转换成指针

4. 若 a 是一个整型变量，则表达式 *&a 等价于：

- A. &a
- B. *a
- C. &&a
- D. a

5. 已定义 `int a = 10;`，下列指针初始化正确的是：

- A. `int *p = a;`
- B. `int *p = &a;`
- C. `int p = &a;`
- D. `int *p = *a;`

6. 下面程序的输出是：

```
int a[] = {1, 2, 3};  
int *p = a;  
printf("%d", *(p + 2));
```

- A. 1
- B. 2
- C. 3
- D. 数组越界

7. 下面程序的输出是：

```
int a[] = {10, 20, 30};  
int *p = a;  
  
printf("%d ", *p++);  
printf("%d", *p);
```

- A. 10 10
- B. 10 20
- C. 20 20
- D. 20 30

8. 执行下面代码后， a 的值是：

```
int a = 5;  
int *p = &a;  
(*p)++;
```

- A. 4
- B. 5
- C. 6
- D. 地址值

9. 下面程序的输出是：

```
int a[] = {5, 10, 15};  
int *p = a;  
printf("%d", *++p);
```

- A. 5
- B. 10
- C. 15
- D. 不确定

10. 若 p 和 q 指向同一数组中的两个元素，则 p - q 表示：

- A. 两个地址相差的字节数
- B. 两个指针所指元素的数值之差

- C. 两个指针之间相差的数组元素个数
- D. 两个指针的地址之和

11. 下列能够正确交换两个整数的函数是：

- A.

```
void swap(int *x, int *y)
{
    int t = *x;
    *x = *y;
    *y = t;
}
```

- B.

```
void swap(int *x, int *y)
{
    int *t = x;
    *x = *y;
    *y = *t;
}
```

- C.

```
void swap(int *x, int *y)
{
    int *t = x;
    x = y;
    y = t;
}
```

- D.

```
void swap(int x, int y)
{
    int t = x;
    x = y;
    y = t;
}
```

12. 在函数参数中，声明 `int a[]` 实际等价于：

- A. `int a`
- B. `int *a`
- C. `int **a`
- D. `int &a`

13. 对于函数：

```
void fun(int a[])  
{  
    printf("%zu", sizeof(a));  
}
```

在函数体内，`sizeof(a)` 得到的是：

- A. 整个数组占用的字节数
- B. 指针变量占用的字节数
- C. 数组元素的个数
- D. 第一个元素占用的字节数

14. 声明“指向一个包含4个整型元素的数组的指针”，正确的是：

- A. `int *p[4];`
- B. `int (*p)[4];`
- C. `int p*[4];`
- D. `int **p[4];`

15. 已定义：

```
int a[3][4];  
int (*p)[4] = a;
```

表达式 `p + 1` 指向：

- A. `a[0][1]`
- B. `a[1][0]`
- C. `a[0][3]`
- D. 整个数组之后

16. 对二维数组 `int a[3][4];`，表达式 `*(*(a + 1) + 2)` 等价于：

- A. `a[0][1]`
- B. `a[1][1]`
- C. `a[1][2]`
- D. `a[2][1]`

17. 下列声明表示“由3个字符指针组成的数组”的是：

- A. `char (*p)[3];`
- B. `char *p[3];`
- C. `char **p[3][3];`
- D. `char p*[3];`

18. 声明 `int *p[5];` 中，`p` 是：

- A. 指向5个整数的指针
- B. 包含5个整型指针的数组
- C. 指向整型数组的指针
- D. 二级指针

19. 声明一个指向函数的指针，该函数有两个 `int` 参数并返回 `int`，正确的是：

- A. `int *p(int, int);`
- B. `int (*p)(int, int);`
- C. `int p*(int, int);`
- D. `int (**p)(int);`

20. 下面程序执行后，`a` 的值是：

```
int a = 3;
int *p = &a;
int **pp = &p;
**pp = 8;
```

- A. 3
- B. 8
- C. `p` 的地址

- D. 不确定

二、字符指针和动态内存

21. 下列字符串定义允许执行 `s[0] = 'A'`；的是：

- A. `char *s = "abc";`
- B. `const char *s = "abc";`
- C. `char s[] = "abc";`
- D. `const char s[] = "abc";`

22. 执行下面代码可能产生的结果是：

```
char *s = "abc";  
s[0] = 'A';
```

- A. 一定得到字符串 "Abc"
- B. 修改字符串字面量，行为未定义
- C. 一定编译失败
- D. s 自动变成字符数组

23. 下面函数的作用是：

```
int fun(char *s)  
{  
    char *p = s;  
  
    while (*p != '\0')  
        p++;  
  
    return p - s;  
}
```

- A. 复制字符串
- B. 比较字符串
- C. 计算字符串长度
- D. 逆置字符串

24. 函数 strcpy(dest, src) 会：

- A. 只复制普通字符，不复制 '\0'
- B. 将 dest 复制到 src
- C. 将 src 连同 '\0' 复制到 dest
- D. 比较两个字符串

25. 执行下面代码后，指针 p 指向：

```
char s[] = "hello";  
char *p = s;
```

```
while (*p != '\0')  
    p++;
```

- A. 字符 'h'
- B. 字符 'o'
- C. 字符串结束标志 '\0'
- D. 数组外的随机位置

26. 手写子串查找函数时，若 a 指向本次匹配的起始位置，begin 指向主串开头，则返回子串起始下标应使用：

- A. p - begin
- B. a - begin
- C. begin - a
- D. *a - *begin

27. 已定义：

```
const int *p;
```

下列说法正确的是：

- A. 不能修改 p 的指向，也不能通过 p 修改 *p
- B. 可以通过 p 修改 *p，不能修改 p 的指向
- C. 不能通过 p 修改 *p，但可以修改 p 的指向
- D. p 和 *p 都可以任意修改

28. 已定义：

```
int *const p = &a;
```

下列说法正确的是：

- A. 可以修改 p 的指向，但不能修改 *p
- B. 不能修改 p 的指向，但可以修改 *p
- C. p 和 *p 都不能修改
- D. p 是指向常量的指针

29. 动态申请 n 个整数空间，推荐的写法是：

- A. `int *p = malloc(n);`
- B. `int *p = malloc(n * sizeof *p);`
- C. `int p = malloc(n * sizeof(int));`
- D. `int *p = malloc(sizeof(p));`

30. malloc() 申请内存失败时返回：

- A. 字符 '0'
- B. -1
- C. NULL
- D. 空字符串

31. 执行 free(p); 后，指针 p：

- A. 自动变成 NULL
- B. 仍保存原地址，但不能再通过该地址合法访问原对象
- C. 仍可继续访问原来的数据
- D. 自动指向下一块内存

32. 使用 realloc() 时更安全的写法是：

- A.

```
p = realloc(p, new_size);
```

- B.

```
int *temp = realloc(p, new_size);
if (temp != NULL)
    p = temp;
```

- C.

```
free(p);
realloc(p, new_size);
```

- D.

```
realloc(NULL, 0);
p++;
```

三、链表

33. 单链表结点结构的正确声明是：

- A.

```
struct Node {
    int data;
    struct Node next;
};
```

- B.

```
struct Node {
    int data;
    struct Node *next;
};
```

- C.

```
struct Node {
    int data;
    int *next;
};
```

- D.

```
struct Node {  
    int data;  
    Node next;  
};
```

34. 若不带头结点的单链表为空，头指针 head 应满足：

- A. head == NULL
- B. head->next == head
- C. head->data == 0
- D. head->next == NULL ，但 head 一定不为空

35. 依次输入 1、2、3 ，每次使用头插法插入链表，最终输出顺序是：

- A. 1 2 3
- B. 3 2 1
- C. 1 3 2
- D. 2 3 1

36. 依次输入 1、2、3 ，使用尾插法建立链表，最终输出顺序是：

- A. 1 2 3
- B. 3 2 1
- C. 2 1 3
- D. 不确定

37. 遍历不带头结点的单链表，能够访问包括尾结点在内的所有结点，正确的循环条件是：

- A. while (p->next != NULL)
- B. while (p != NULL)
- C. while (*p != NULL)
- D. while (p == NULL)

38. 已知 p 指向链表中的某个结点，要将 newnode 插入到 p 后面，正确的是：

- A.

```
p->next = newnode;  
newnode->next = p->next;
```

- B.

```
newnode->next = p->next;  
p->next = newnode;
```

- C.

```
newnode = p->next;  
p->next = newnode;
```

- D.

```
p = newnode;  
newnode->next = NULL;
```

39. 已知 p->next 是待删除结点，正确的删除方法是：

- A.

```
free(p->next);  
p->next = p->next->next;
```

- B.

```
Node *q = p->next;  
p->next = q->next;  
free(q);
```

- C.

```
p->next = NULL;
```

- D.

```
free(p);
```

40. 删除不带头结点链表的第一个结点时，函数参数使用 `Node **head` 的主要原因是：

- A. 节省内存
- B. 可以修改调用函数中的头指针
- C. 可以避免使用 `free()`
- D. 可以把链表变成数组

41. 删除不带头结点链表的首结点，正确代码是：

- A.

```
Node *q = *head;  
*head = q->next;  
free(q);
```

- B.

```
free(*head);  
*head = (*head)->next;
```

- C.

```
*head = NULL;
```

- D.

```
free(head);
```

42. 删除链表结点时，如果只修改指针而不调用 `free()`，可能造成：

- A. 数组越界
- B. 内存泄漏
- C. 编译错误
- D. 链表自动形成环

43. 遍历一个包含 n 个结点的单链表，其时间复杂度是：

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n)$
- D. $O(n^2)$

44. 原地逆置单链表时，正确的核心操作顺序是：

- A.

```
next = p->next;
p->next = previous;
previous = p;
p = next;
```

- B.

```
p = p->next;
p->next = previous;
previous = p;
```

- C.

```
free(p);
p = next;
```

- D.

```
previous->next = p;
p = p->next;
```

45. 使用快慢指针寻找链表中间结点时，通常：

- A. 快指针每次走一步，慢指针每次走两步
- B. 快指针每次走两步，慢指针每次走一步
- C. 两个指针都不移动
- D. 两个指针始终每次走两步

46. 使用双指针查找倒数第 k 个结点时，正确思路是：

- A. 两个指针始终指向同一结点

- B. 快指针先走 k 步，然后快慢指针同时移动
- C. 慢指针先走 k 步
- D. 必须先逆置链表

47. 两个有序链表分别有 m 和 n 个结点，将它们合并为一个有序链表的时间复杂度是：

- A. $O(1)$
- B. $O(\log(m+n))$
- C. $O(m+n)$
- D. $O(mn)$

48. 判断单链表是否存在环，常用的方法是：

- A. 冒泡排序
- B. 快慢指针
- C. 二分查找
- D. 头插法

49. 假设循环单链表非空，对其进行一轮完整遍历，合适的结构是：

- A.

```
while (p != NULL) {
    p = p->next;
}
```

- B.

```
p = head;
do {
    printf("%d", p->data);
    p = p->next;
} while (p != head);
```

- C.

```
while (p == head) {
    p = NULL;
}
```

- D.

```
for (;;) {  
    free(p);  
}
```

50. 使用链表表示多项式，当两个结点的指数相同时，进行多项式加法应当：

- A. 将指数相加
- B. 将系数相加，指数保持不变
- C. 同时删除两个结点
- D. 将系数相乘

答题卡

1. __	2. __	3. __	4. __	5. __
6. __	7. __	8. __	9. __	10. __
11. __	12. __	13. __	14. __	15. __
16. __	17. __	18. __	19. __	20. __
21. __	22. __	23. __	24. __	25. __
26. __	27. __	28. __	29. __	30. __
31. __	32. __	33. __	34. __	35. __
36. __	37. __	38. __	39. __	40. __
41. __	42. __	43. __	44. __	45. __
46. __	47. __	48. __	49. __	50. __

参考答案

建议完成全部题目后再展开答案。

► [点击查看参考答案](#)